

Implementacion y Analisis de Complejidad de Listas, Pilas y Colas en Java

Estudiante: Marco Antonio Garcia Villasmil

Codigo: 1094050756

Profesor: David Herrera **Monitora:** Angela Camila Siabato Londono

Repositorio: <https://github.com/marcogarciavillasmil/lab-listas-pilas-colas>

1. Objetivo del taller

El objetivo de este taller es implementar en Java la estructura List usando listas enlazadas en sus cuatro variantes (simple sin cola, simple con cola, doble sin cola y doble con cola), y las estructuras Stack y Queue usando arreglo dinamico y arreglo circular. Con las 8 implementaciones ya funcionando, el trabajo real del taller es medir el tiempo de ejecucion de cada metodo para distintos tamanos de entrada, comparar esos tiempos contra la complejidad teorica en notacion Big-O, y sacar conclusiones sobre quien conviene usar en cada caso.

2. Explicacion de la implementacion

2.1 List (listas enlazadas)

Se implemento una interfaz comun `MyList<T>` con los 8 metodos que pide el enunciado (`pushFront`, `pushBack`, `popFront`, `popBack`, `find`, `erase`, `addBefore`, `addAfter`), y 4 clases que la implementan:

- **SinglyLinkedListNoTail:** solo guarda head. Todo lo que necesite llegar al final (`pushBack`, `popBack`, `topBack`) o encontrar el nodo anterior a otro (`erase`, `addBefore`) tiene que recorrer la lista completa.
- **SinglyLinkedListWithTail:** agrega un puntero tail. `pushBack` pasa a ser $O(1)$, pero `popBack` sigue siendo $O(n)$ porque los nodos no tienen puntero al anterior (es simple, no doble) y hay que recorrer para encontrar quien queda como nuevo tail.
- **DoublyLinkedListNoTail:** los nodos ya tienen prev y next, asi que dado un nodo, `erase/addBefore/addAfter` son $O(1)$. El problema es que sin tail, llegar al ultimo nodo sigue siendo $O(n)$.
- **DoublyLinkedListWithTail:** head + tail + prev/next en cada nodo. Es la version completa: todo queda en $O(1)$ menos `find`, que al no haber indexacion siempre es $O(n)$.

`find()` devuelve la referencia al Node (no un booleano ni un indice) para poder pasarla despues a `erase`, `addBefore` o `addAfter` sin tener que volver a buscar el elemento.

2.2 MyStack y MyQueue

Para MyStack y MyQueue se implemento cada una con dos estrategias de arreglo:

- **Arreglo dinamico (ArrayStack / ArrayQueue):** arreglo interno que se duplica cuando se llena, igual que ArrayList. En el Stack esto alcanza para que push/pop sean $O(1)$ amortizado porque ambos operan sobre el mismo extremo (el tope). En el Queue, en cambio, enqueue mete al final pero dequeue tiene que sacar la posicion 0 y correr todos los elementos restantes una posicion, lo que deja dequeue en $O(n)$.
- **Arreglo circular (CircularArrayStack / CircularArrayQueue):** en vez de un arreglo plano, se maneja un indice front y las posiciones se calculan modulo la capacidad. Para el Stack esto no cambia nada frente a la version dinamica (sigue operando en un solo extremo), pero para el Queue es la diferencia clave: dequeue deja de desplazar elementos, solo avanza el indice front, quedando en $O(1)$.

El crecimiento del arreglo (cuando se llena) se hace duplicando la capacidad (capacidad $\ast = 2$), la estrategia estandar de amortizacion: el costo total de n inserciones sigue siendo $O(n)$, aunque una insercion puntual pueda costar $O(n)$ cuando toca redimensionar.

2.3 Medicion de tiempos

La medicion sigue el mismo patron del Main.java de referencia (Instant/Duration), pero usando `System.nanoTime()` y reportando en microsegundos: en listas chicas, milisegundos redondeaba todo a 0 y no se alcanzaba a notar la diferencia entre un metodo $O(1)$ y uno $O(n)$. Para cada tamano n se arma primero la estructura con n elementos (usando siempre un metodo $O(1)$ para no distorsionar el tiempo de armado), y despues se repite el metodo que se quiere medir varias veces seguidas, promediando. En push/pop se deshace cada operacion inmediatamente despues de medirla para que el tamano de la estructura se mantenga estable durante toda la medicion. En metodos que buscan un valor (find, erase, addBefore, addAfter, delete) el objetivo se elige al azar (semilla fija = 42, para que sea reproducible) entre los valores existentes.

Los tamanos usados fueron $n = 10, 100, 1000, 10\ 000, 100\ 000$ y $1\ 000\ 000$. El enunciado sugiere llegar hasta 10^8 , pero para los metodos $O(n)$ eso significa que una sola corrida del benchmark tarda minutos, asi que se decidio limitar a 10^6 : en las graficas se ve con claridad la tendencia (recta ascendente en escala log-log para $O(n)$, plana para $O(1)$), y extrapolar dos ordenes de magnitud mas no cambia la conclusion.

3. Analisis de complejidad

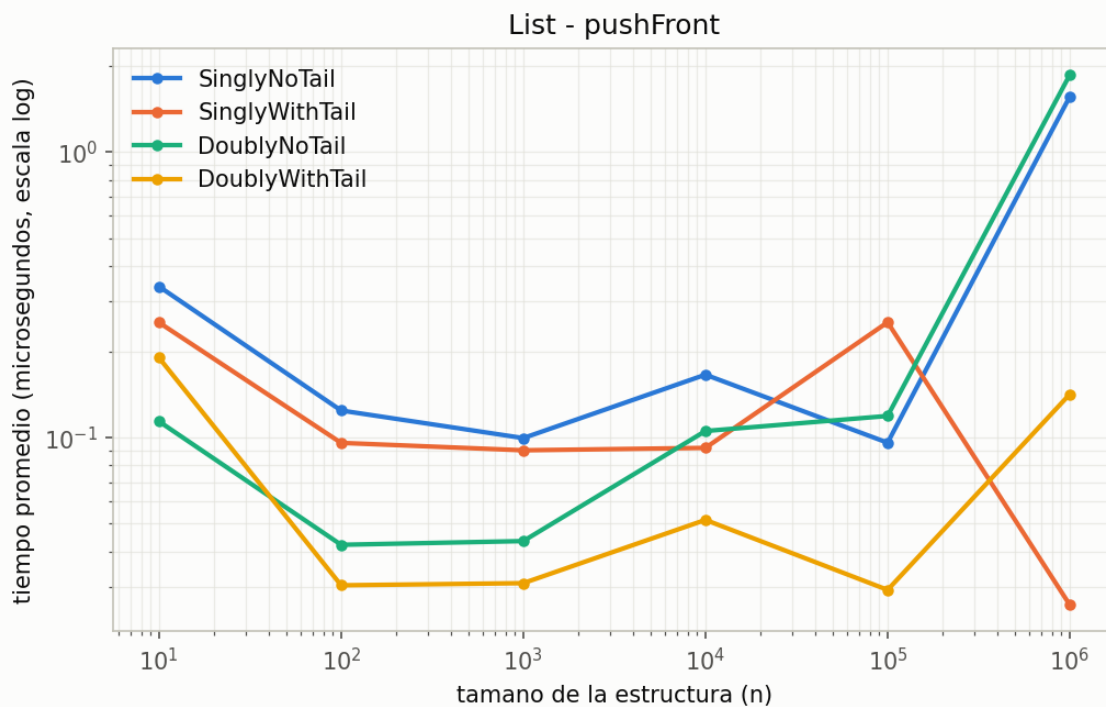
3.1 List: complejidad teorica

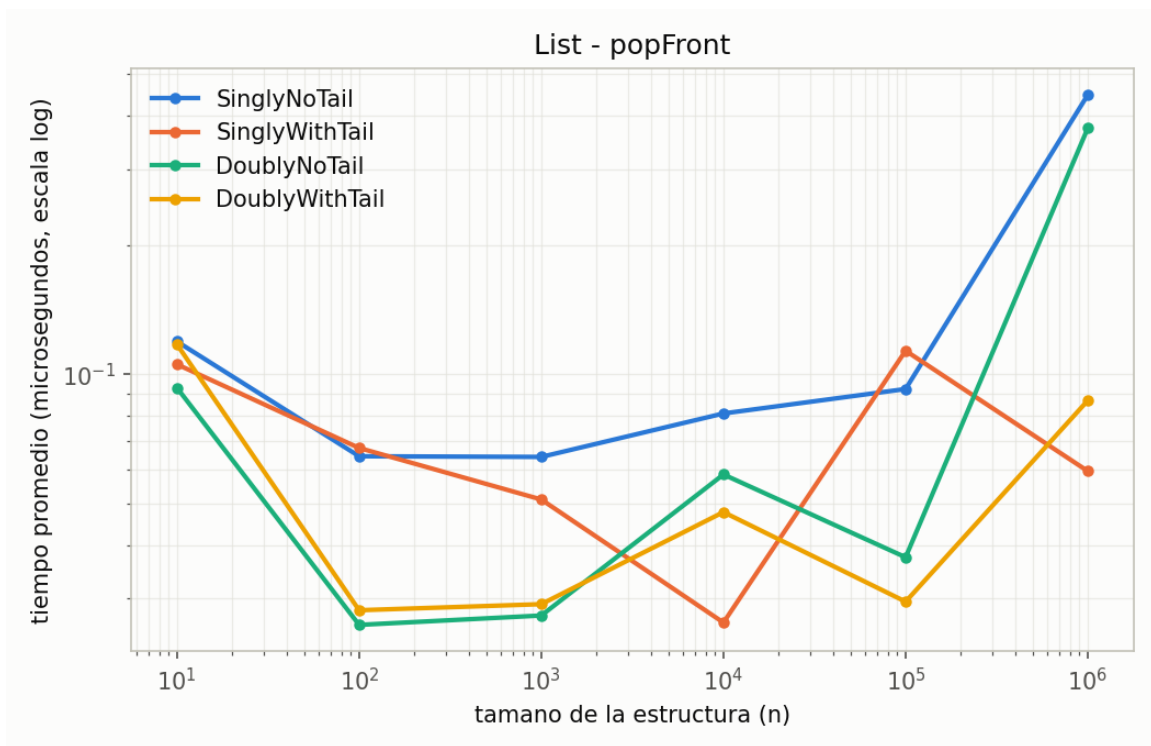
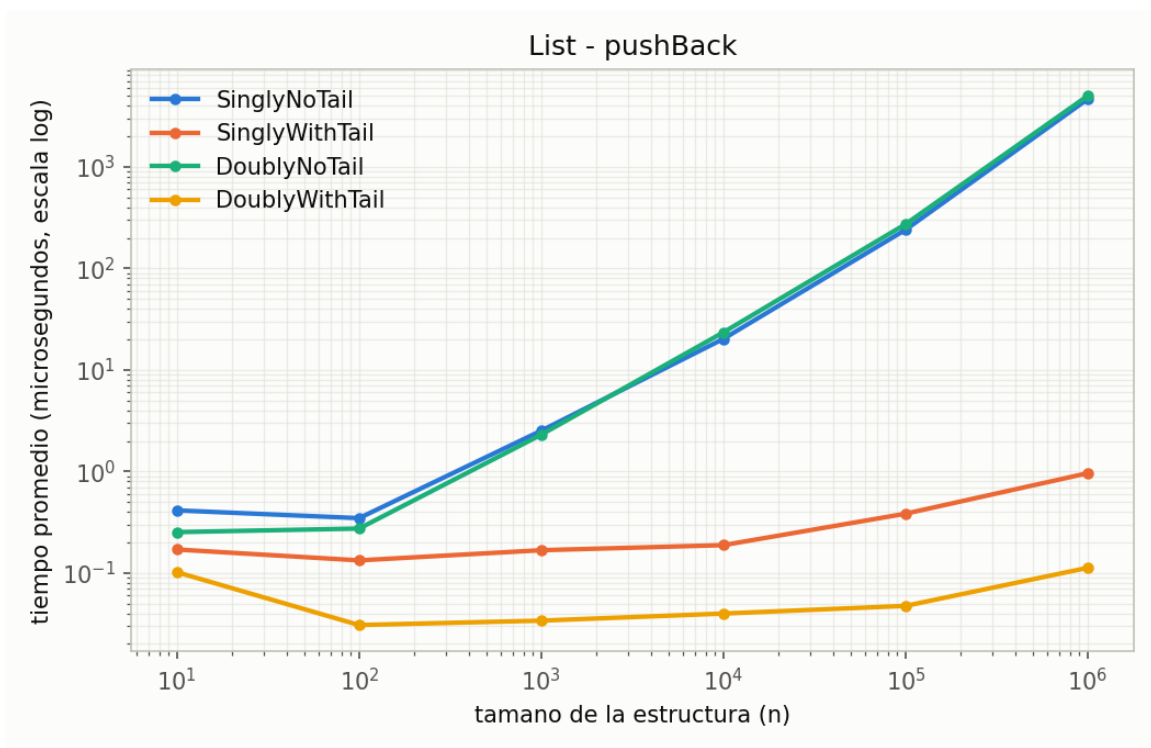
metodo	SinglyNoTail	SinglyWithTail	DoublyNoTail	DoublyWithTail
pushFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
pushBack	$O(n)$	$O(1)$	$O(n)$	$O(1)$
popFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
popBack	$O(n)$	$O(n)$	$O(n)$	$O(1)$
find	$O(n)$	$O(n)$	$O(n)$	$O(n)$
erase	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addBefore	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addAfter	$O(1)$	$O(1)$	$O(1)$	$O(1)$

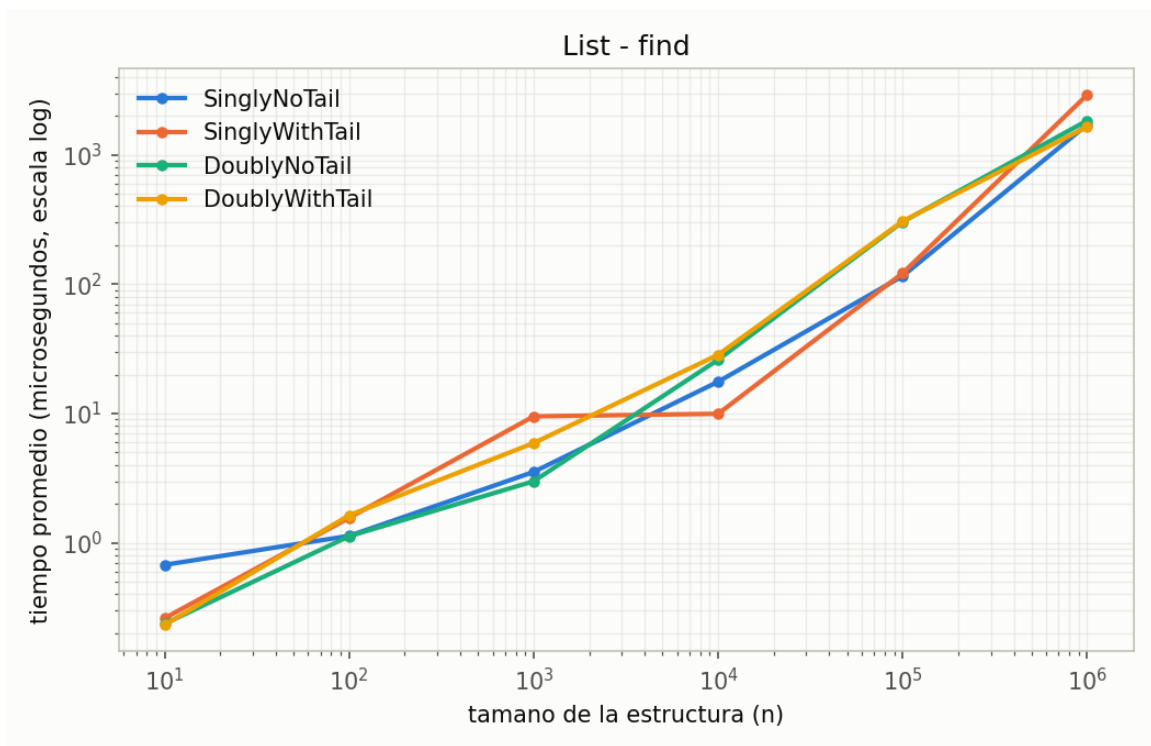
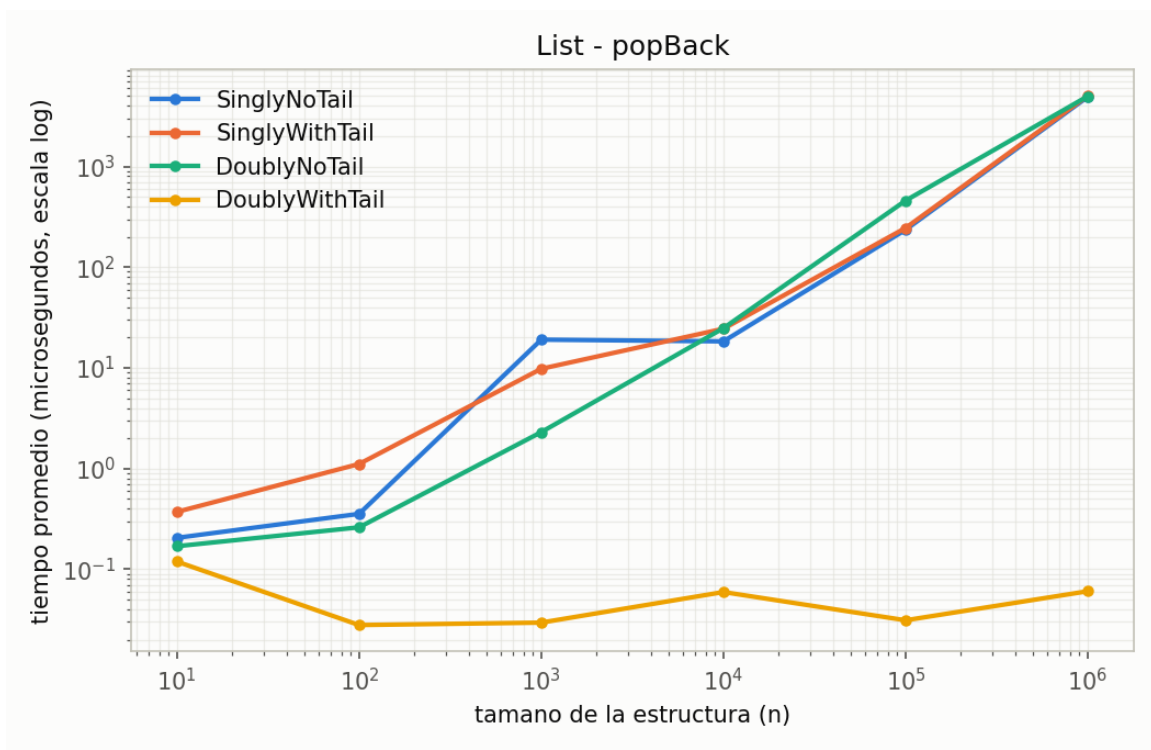
popBack en SinglyWithTail queda en $O(n)$ a pesar de tener tail porque, al ser simple, no hay forma de llegar al penultimo nodo sin recorrer desde el head. erase y addBefore necesitan el nodo anterior al indicado: en las listas simples eso obliga a recorrer, en las dobles no porque cada nodo ya conoce a su vecino.

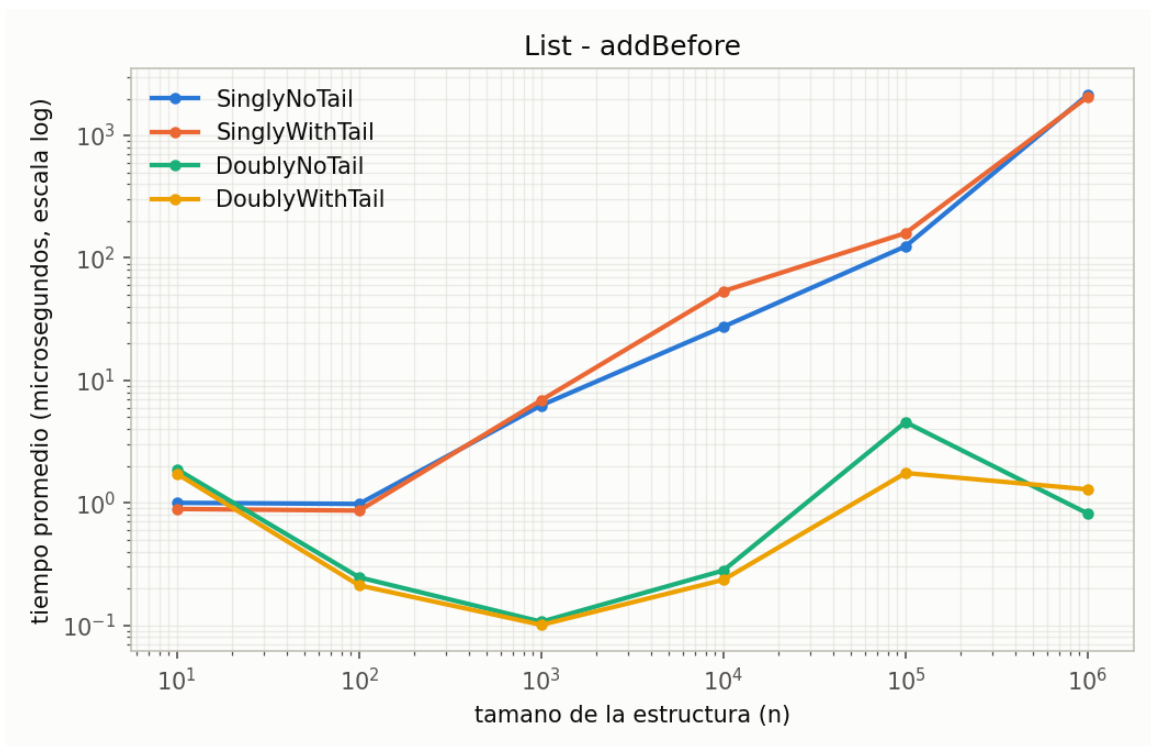
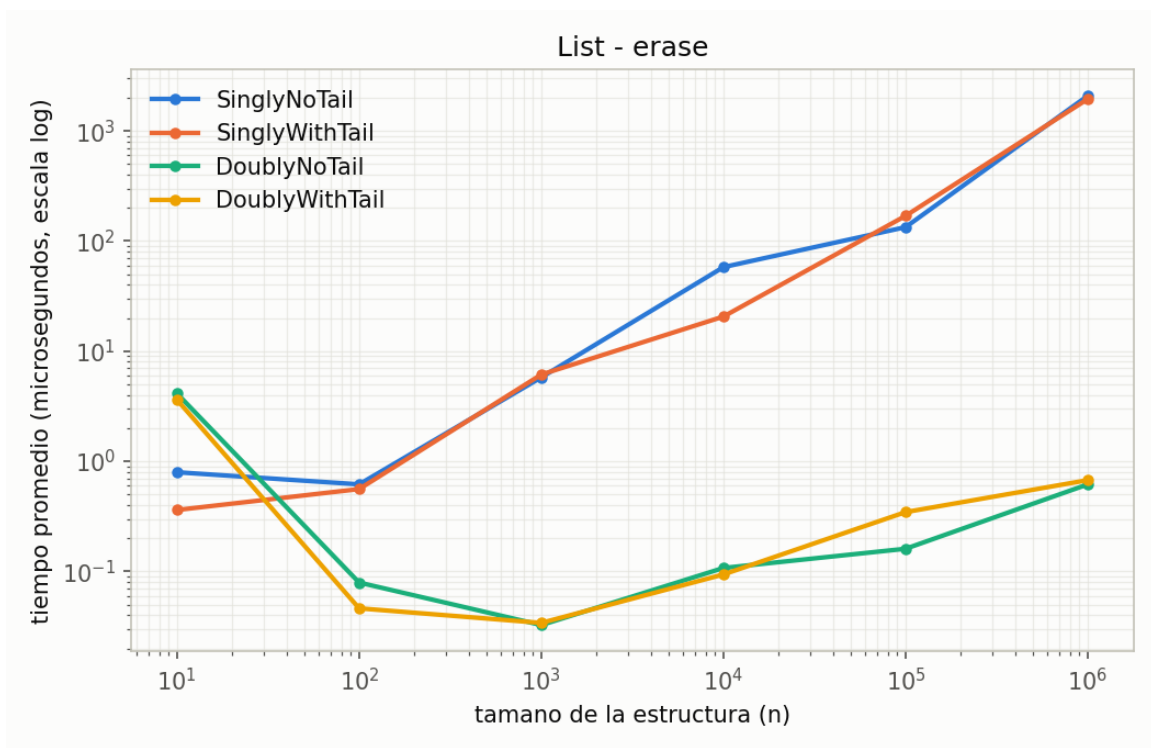
3.2 List: resultados empiricos por metodo

Tiempos promedio en microsegundos, uno por metodo (las 4 implementaciones juntas para comparar):

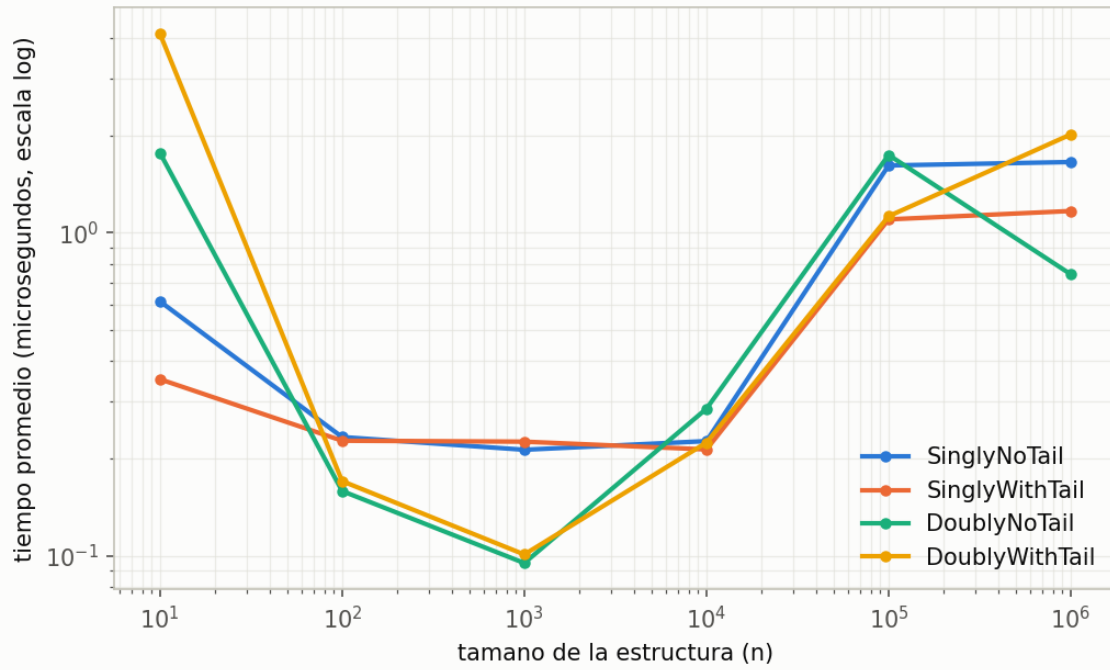








List - addAfter



3.3 Tabla de resultados - SinglyLinkedListNoTail

n	10	100	1000	10000	100000	1000000
pushFront	0.34	0.12	0.10	0.17	0.10	1.56
pushBack	0.41	0.35	2.52	20.25	240.65	4664.91
popFront	0.12	0.06	0.06	0.08	0.09	0.45
popBack	0.21	0.36	19.08	18.34	235.37	4923.88
find	0.68	1.13	3.53	17.61	115.51	1712.92
erase	0.80	0.62	5.78	57.97	133.73	2086.53
addBefore	1.00	0.98	6.25	27.42	124.48	2146.79
addAfter	0.61	0.23	0.21	0.23	1.61	1.65

Tabla de resultados - SinglyLinkedListWithTail

n	10	100	1000	10000	100000	1000000
pushFront	0.25	0.10	0.09	0.09	0.25	0.03
pushBack	0.17	0.13	0.17	0.19	0.38	0.96
popFront	0.11	0.07	0.05	0.03	0.11	0.06
popBack	0.37	1.11	9.80	24.48	245.54	5046.88
find	0.26	1.56	9.51	9.95	121.69	2907.83
erase	0.36	0.56	6.13	20.61	169.43	1948.91
addBefore	0.89	0.86	6.89	53.51	159.46	2068.60
addAfter	0.35	0.23	0.23	0.21	1.10	1.17

Tabla de resultados - DoublyLinkedListNoTail

n	10	100	1000	10000	100000	1000000
pushFront	0.11	0.04	0.04	0.11	0.12	1.86
pushBack	0.25	0.27	2.30	23.47	273.03	5024.43
popFront	0.09	0.03	0.03	0.06	0.04	0.37
popBack	0.17	0.26	2.30	24.80	458.77	4992.75
find	0.24	1.13	2.99	26.05	303.68	1838.96
erase	4.13	0.08	0.03	0.11	0.16	0.62
addBefore	1.88	0.25	0.11	0.28	4.56	0.82
addAfter	1.77	0.16	0.09	0.29	1.74	0.74

Tabla de resultados - DoublyLinkedListWithTail

n	10	100	1000	10000	100000	1000000
---	----	-----	------	-------	--------	---------

pushFront	0.19	0.03	0.03	0.05	0.03	0.14
pushBack	0.10	0.03	0.03	0.04	0.05	0.11
popFront	0.12	0.03	0.03	0.05	0.03	0.09
popBack	0.12	0.03	0.03	0.06	0.03	0.06
find	0.23	1.63	5.91	28.45	306.48	1647.81
erase	3.62	0.05	0.03	0.09	0.35	0.67
addBefore	1.73	0.21	0.10	0.23	1.75	1.29
addAfter	4.13	0.17	0.10	0.22	1.12	2.01

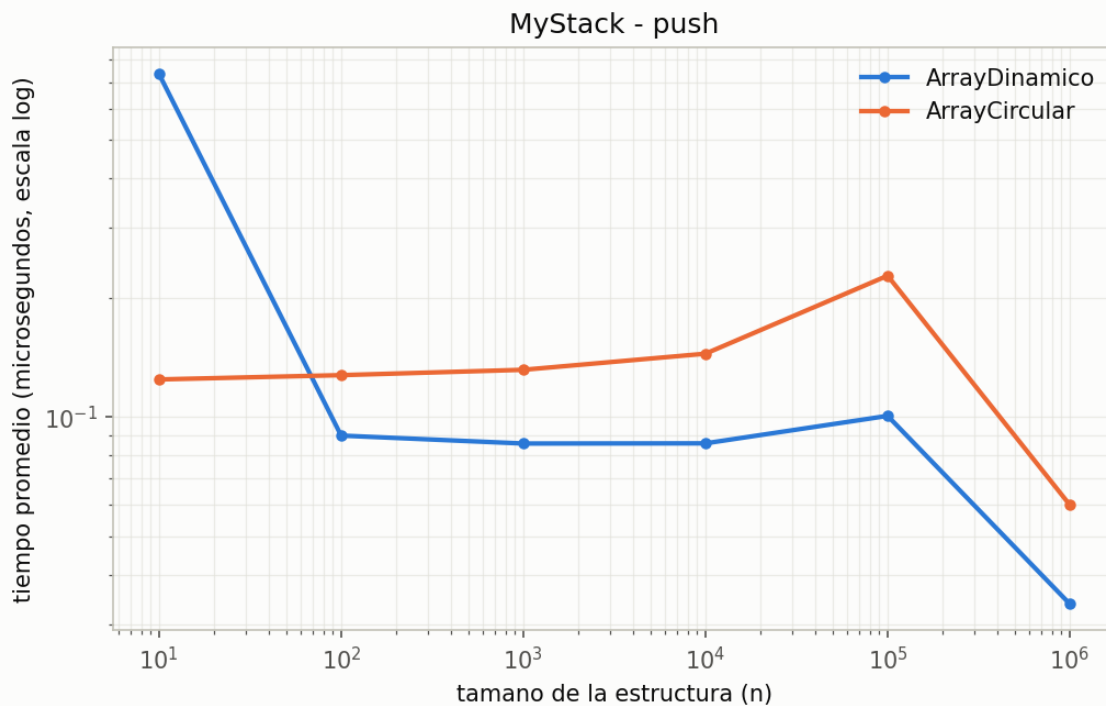
Los resultados coinciden con lo teorico: pushBack y popBack se disparan en las implementaciones sin tail (o sin tail util, caso popBack en SinglyWithTail), find crece en las 4 por igual, y erase/addBefore se mantienen practicamente planos solo en las listas dobles. addAfter es $O(1)$ en las 4, y eso tambien se ve reflejado.

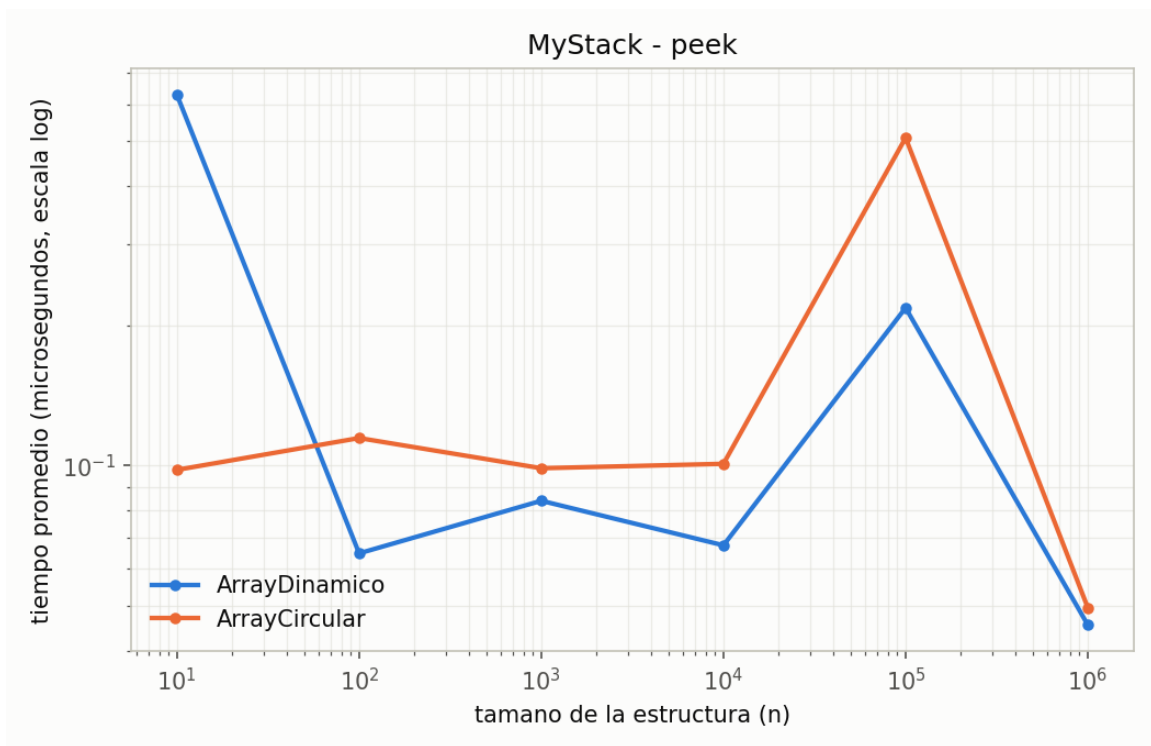
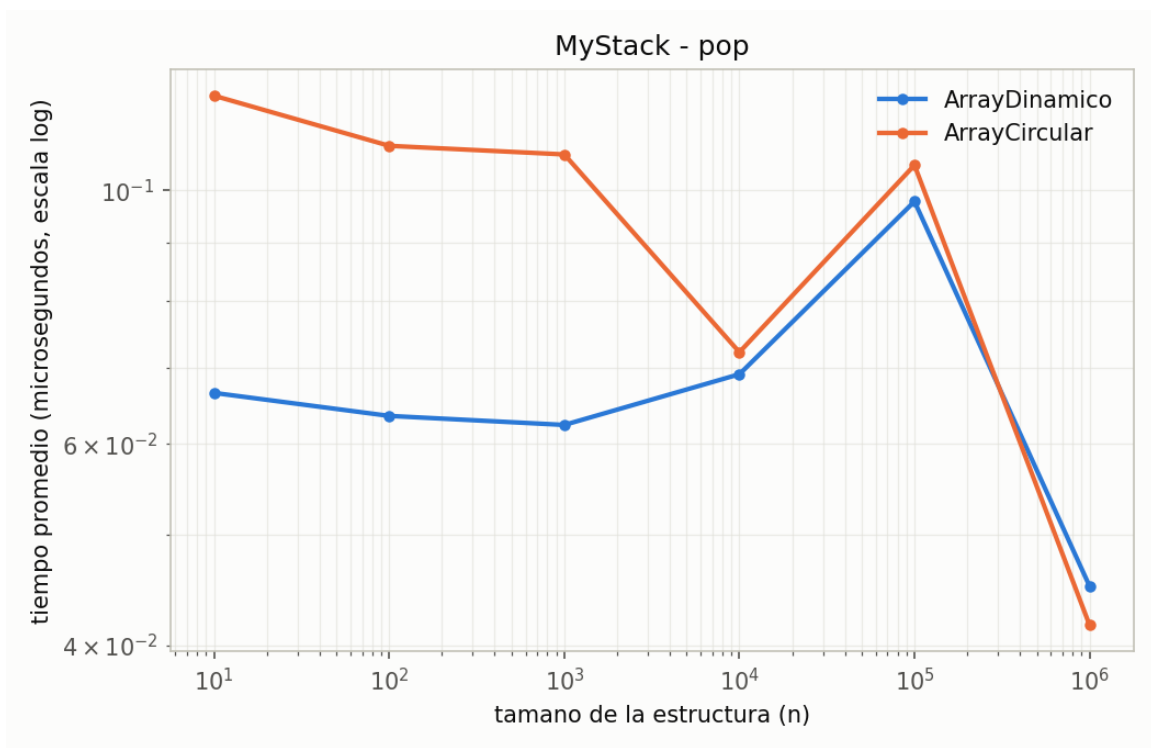
3.4 MyStack y MyQueue: complejidad teorica

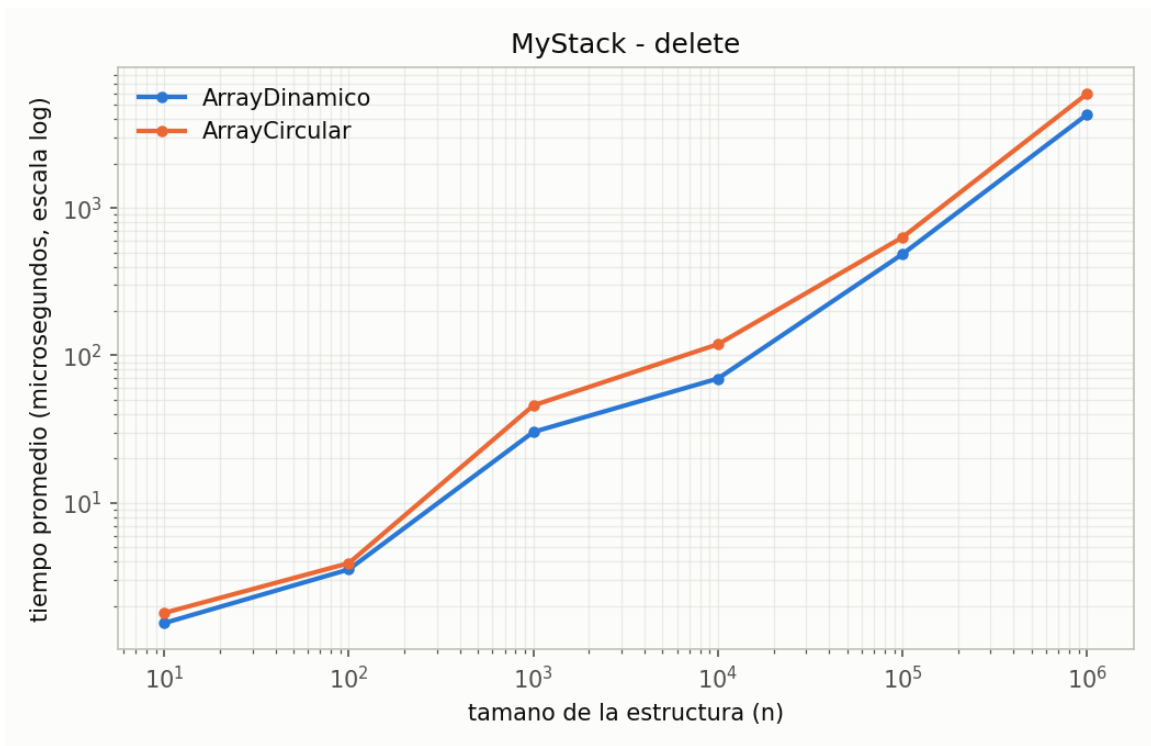
operacion	Arreglo dinamico	Arreglo circular
push / enqueue	$O(1)$ am.	$O(1)$ am.
pop / dequeue (Stack)	$O(1)$	$O(1)$
dequeue (Queue)	$O(n)$	$O(1)$
peek / front	$O(1)$	$O(1)$
delete(valor)	$O(n)$	$O(n)$

Para el Stack, push y pop operan siempre sobre el mismo extremo (el tope), así que el arreglo circular no aporta nada frente al dinámico: ambos quedan $O(1)$. La diferencia real aparece en el Queue: con arreglo dinámico, dequeue saca el elemento 0 y desplaza todo el resto ($O(n)$); con arreglo circular solo se mueve el índice front ($O(1)$).

3.5 MyStack: resultados empiricos





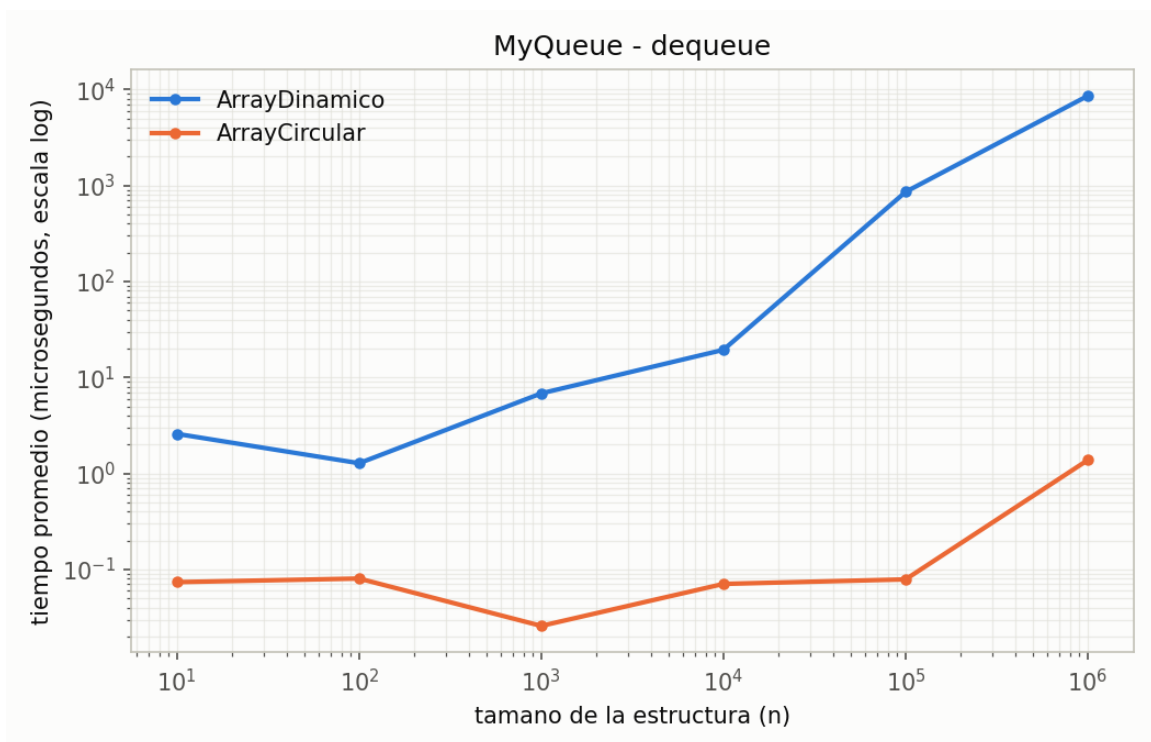
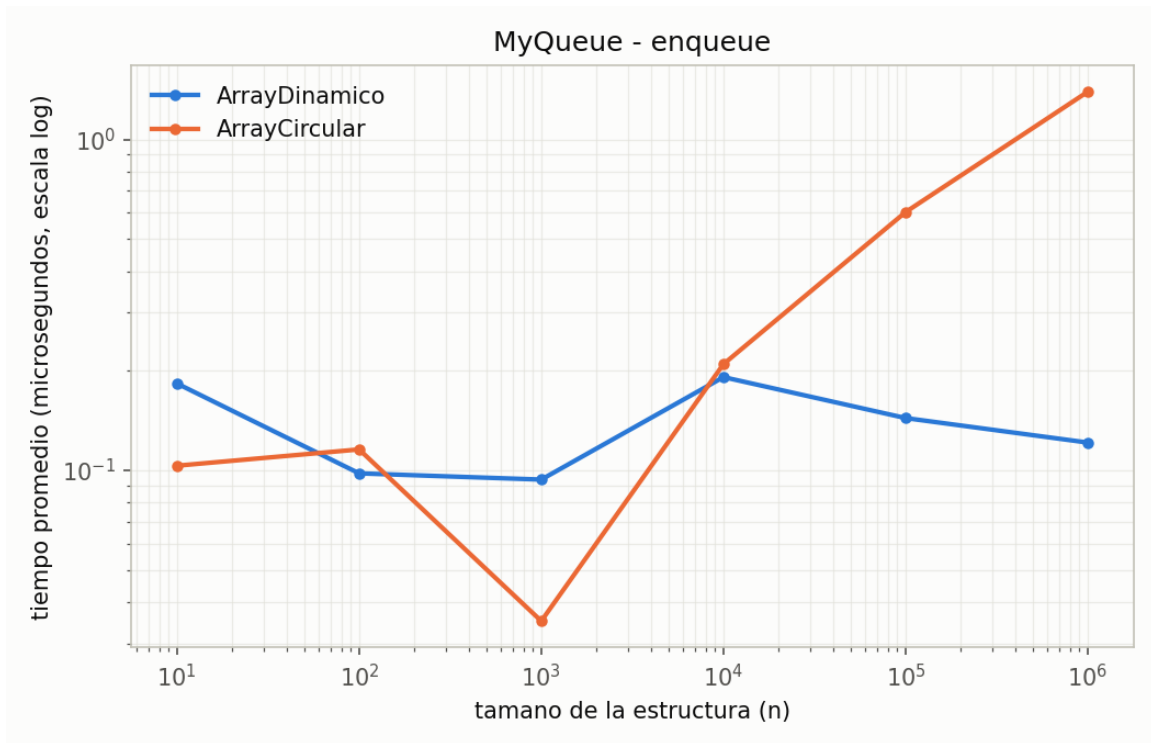


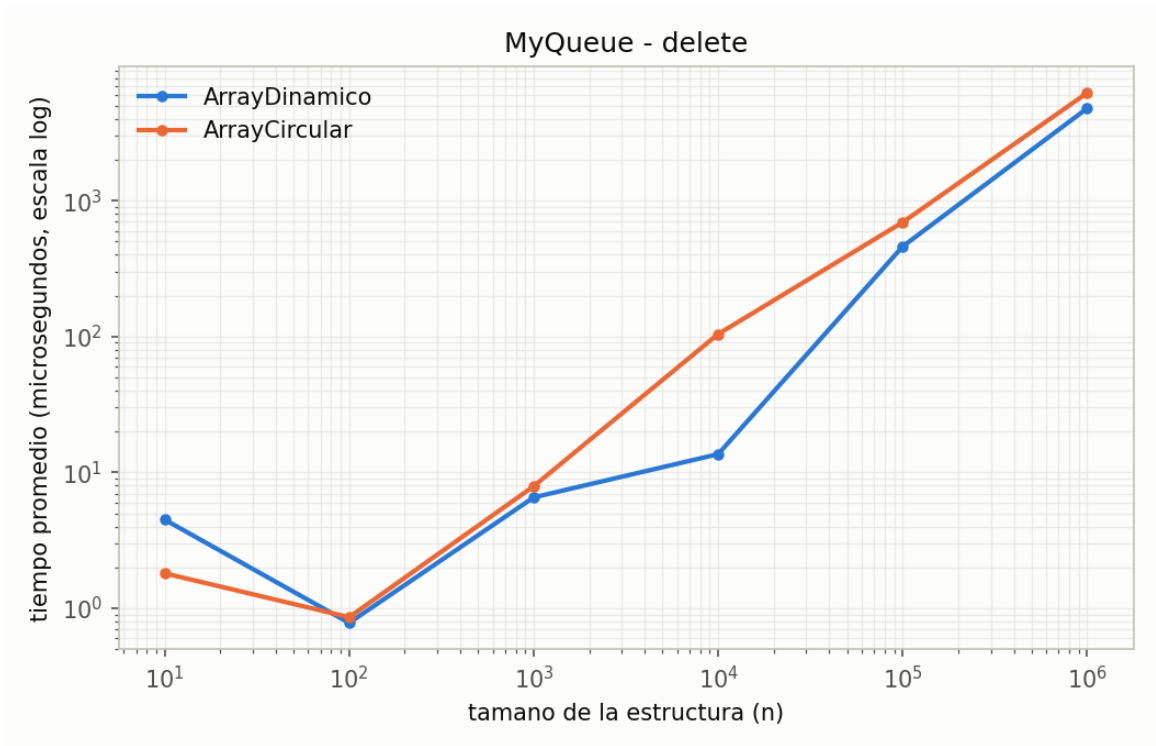
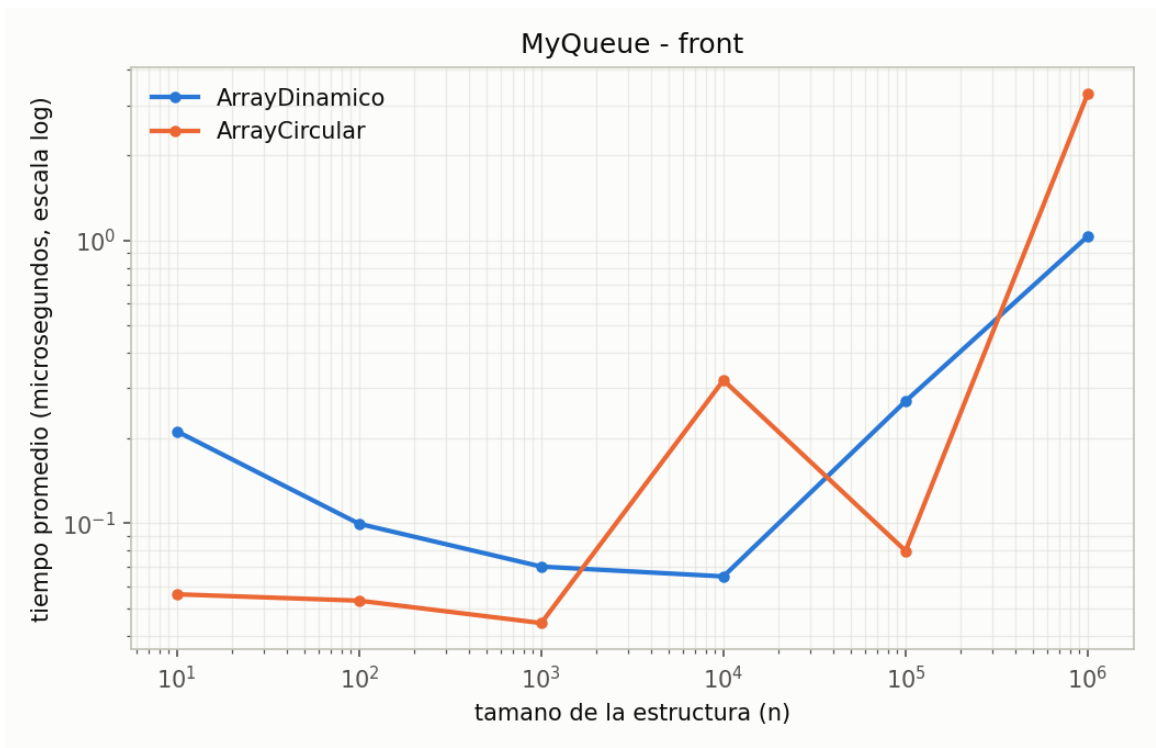
n	10	100	1000	10000	100000	1000000
push	0.73	0.09	0.09	0.09	0.10	0.03
pop	0.07	0.06	0.06	0.07	0.10	0.05
peek	0.63	0.06	0.08	0.07	0.22	0.05
delete	1.53	3.54	30.27	69.68	486.63	4303.57

n	10	100	1000	10000	100000	1000000
push	0.12	0.13	0.13	0.14	0.23	0.06
pop	0.12	0.11	0.11	0.07	0.11	0.04
peek	0.10	0.11	0.10	0.10	0.51	0.05
delete	1.80	3.90	45.82	119.03	632.77	5943.94

Tal como se esperaba, push/pop/peek quedan practicamente iguales entre las dos implementaciones del Stack -- las dos curvas se superponen en las graficas. delete crece igual en ambas porque en las dos hay que recorrer buscando el valor.

3.6 MyQueue: resultados empiricos





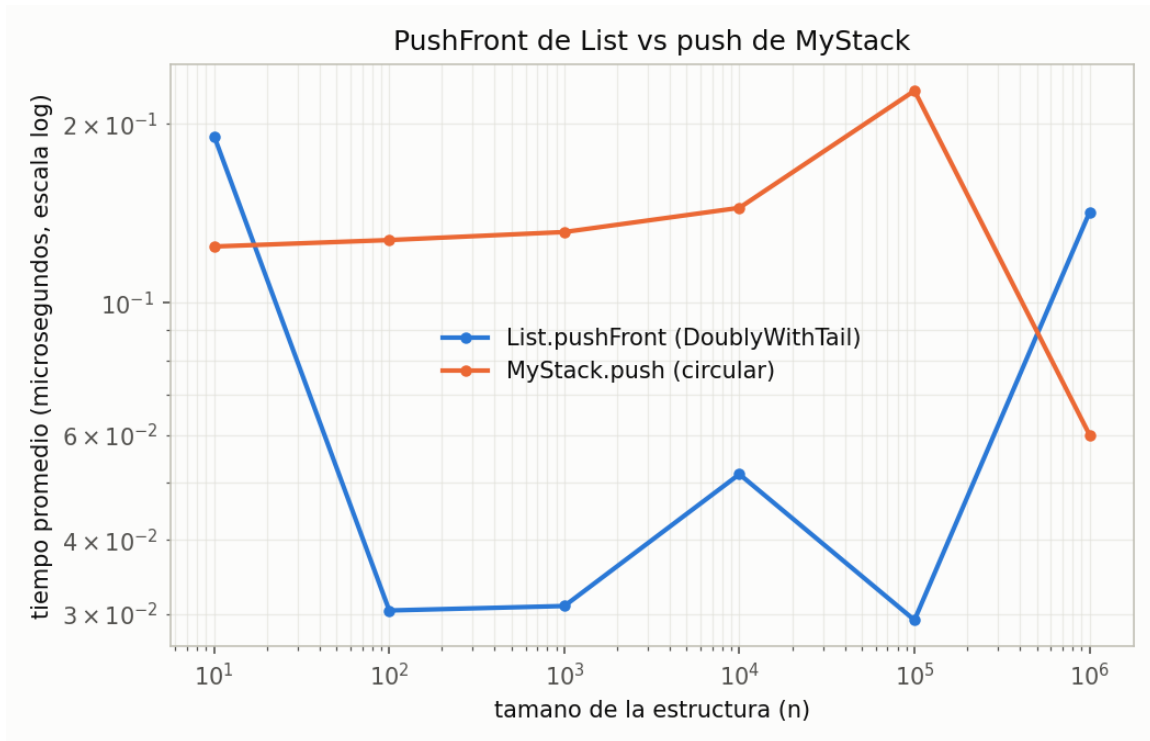
n	10	100	1000	10000	100000	1000000
enqueue	0.18	0.10	0.09	0.19	0.14	0.12
dequeue	2.58	1.28	6.83	19.41	858.27	8594.46
front	0.21	0.10	0.07	0.06	0.27	1.03
delete	4.50	0.78	6.55	13.63	459.82	4778.73

n	10	100	1000	10000	100000	1000000
enqueue	0.10	0.12	0.04	0.21	0.60	1.39
dequeue	0.07	0.08	0.03	0.07	0.08	1.38
front	0.06	0.05	0.04	0.32	0.08	3.30
delete	1.81	0.86	7.92	104.30	692.92	6209.17

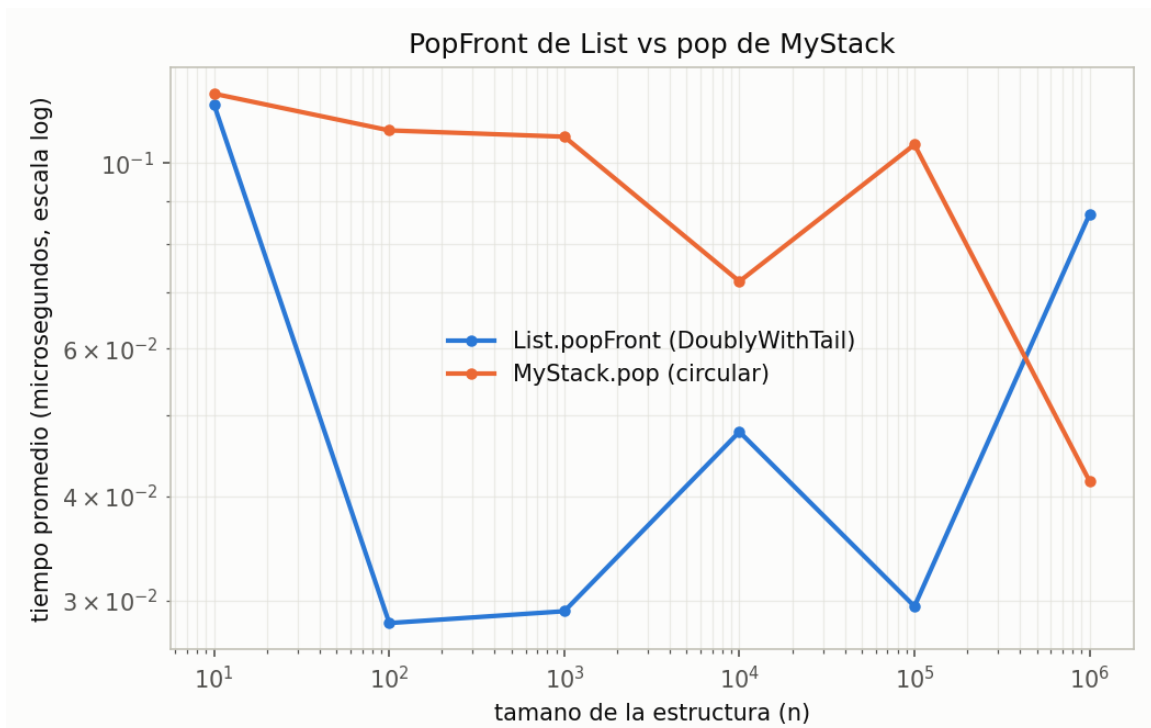
Aquí se nota la diferencia: dequeue en ArrayDinámico crece de forma clara con n (llega a más de 8500 microsegundos en $n=1\,000\,000$), mientras que en ArrayCircular se mantiene prácticamente plano en todo el rango. Es la mejor evidencia empírica de todo el taller de que el arreglo circular vale la pena cuando se necesita sacar elementos por el frente.

3.7 Comparativa de metodos equivalentes (List vs MyStack/MyQueue)

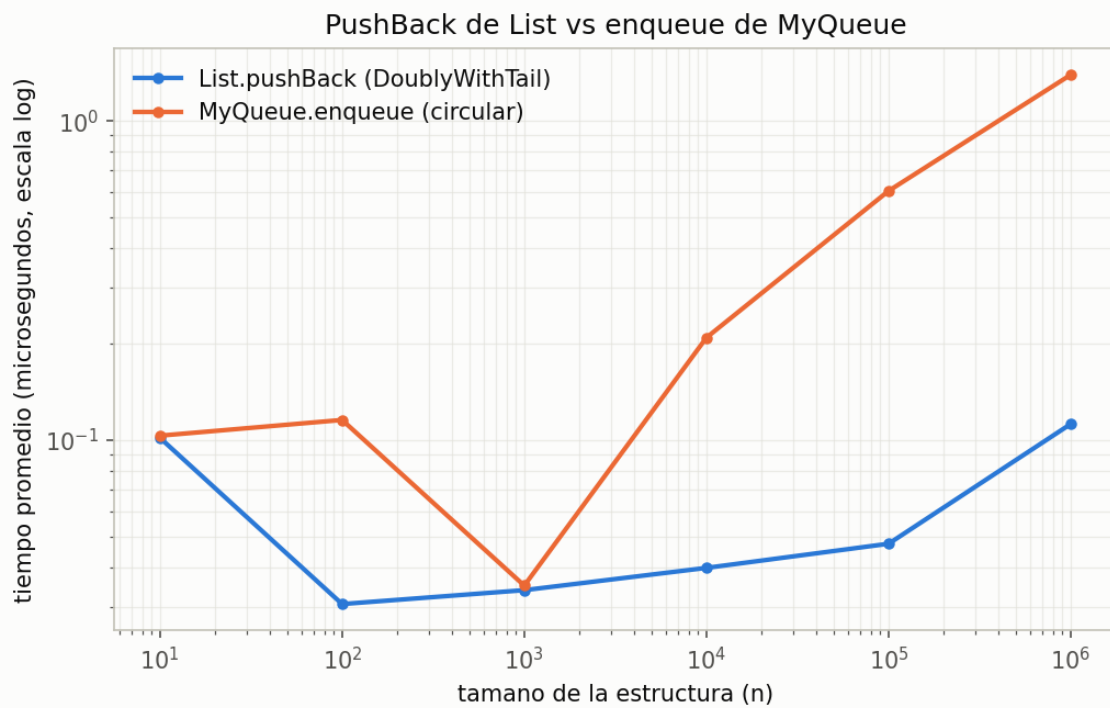
Para esta comparacion se eligio siempre **DoublyLinkedListWithTail** como representante de List. La razon es que, segun la tabla teorica de 3.1, es la unica de las 4 implementaciones que queda en $O(1)$ en todos los metodos relevantes para esta comparacion (pushFront, popFront, pushBack, erase); las otras 3 tienen al menos un metodo en $O(n)$, asi que comparar con ellas hubiera sido comparar la version circular contra "la peor version posible" de List, lo cual no tendria sentido para decidir cual estructura conviene usar. Se compara siempre contra la version de **arreglo circular** de MyStack/MyQueue por ser tambien la mas optima de sus dos implementaciones.



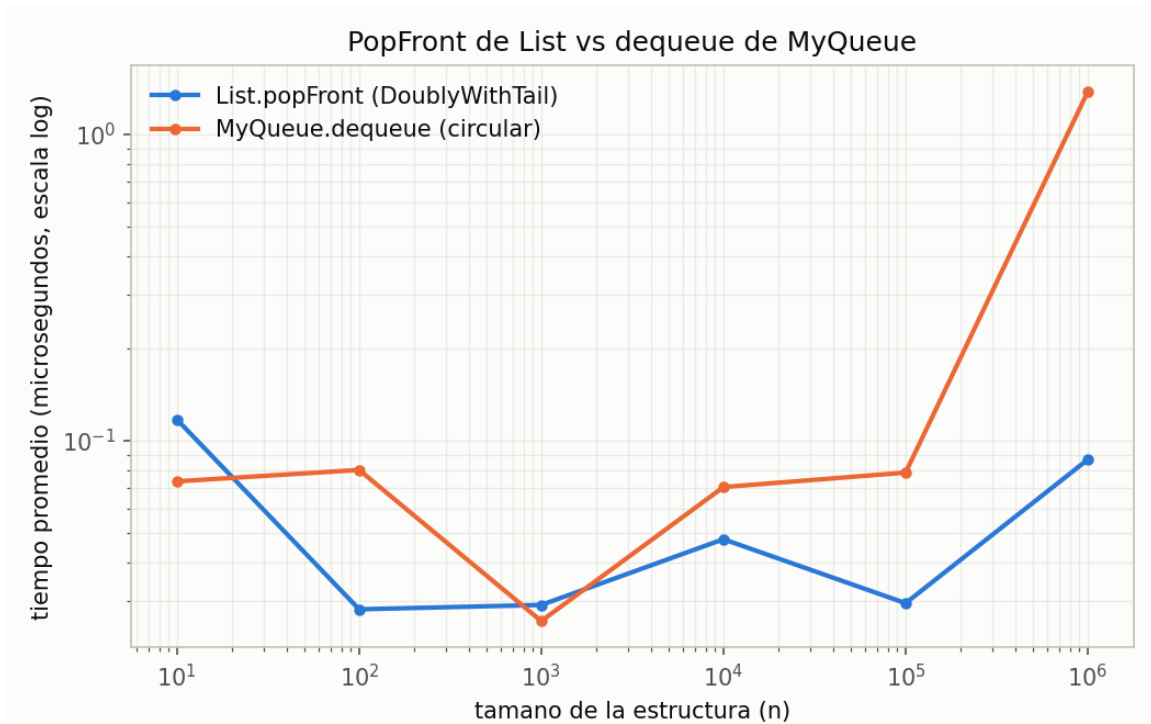
PushFront de List (uso tipico como pila) vs push de MyStack.



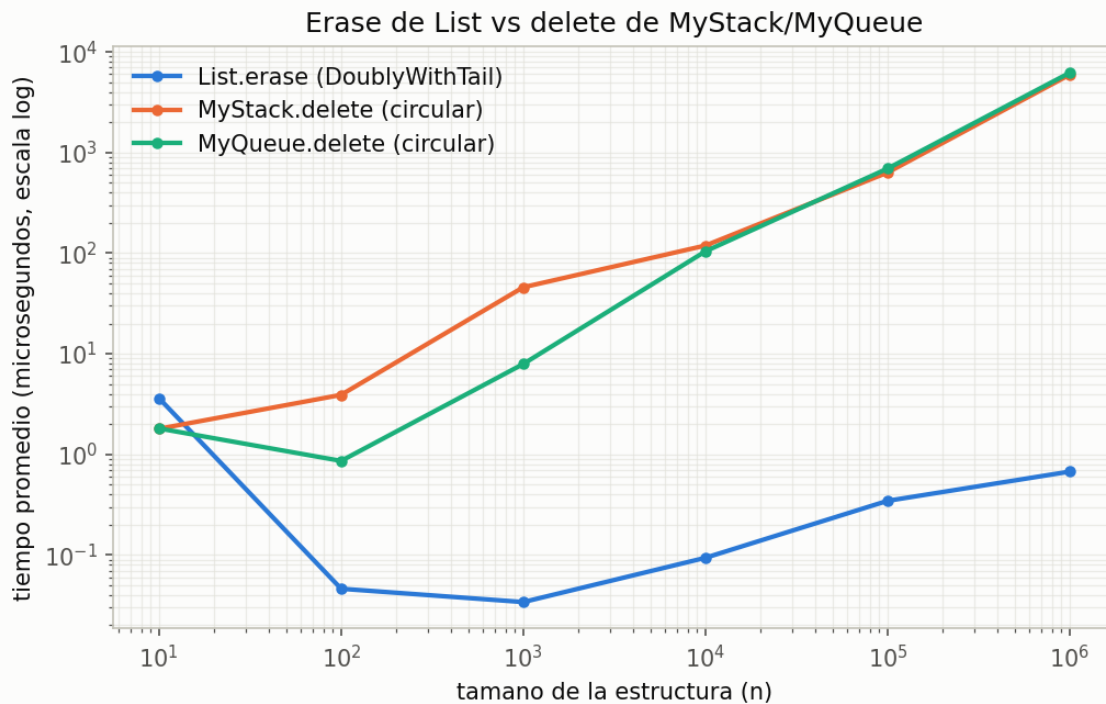
PopFront de List vs pop de MyStack.



PushBack de List (uso tipico como cola) vs enqueue de MyQueue.



PopFront de List vs dequeue de MyQueue.



Erase de List vs delete de MyStack y MyQueue (busqueda + eliminacion).

En push/pop puros las tres estructuras quedan practicamente empatadas -- las dos son $O(1)$, la diferencia esta solo en la constante (la lista paga malloc de un nodo por cada insercion; el arreglo circular a veces paga un resize). En delete, en cambio, List sale ganando frente a Stack/Queue con arreglo: aunque las tres son $O(n)$ porque hay que buscar el valor, en la lista el borrado en si mismo es $O(1)$ una vez encontrado (no hay

que desplazar nada), mientras que en el arreglo, además de buscar, hay que correr los elementos para cerrar el hueco.

4. Conclusiones

- **Ninguna de las 4 implementaciones de List es la mejor en todo.** SinglyLinkedListNoTail es la mas simple de programar pero es la que peor escala: pushBack, popBack, erase y addBefore son $O(n)$. Si solo se necesitan operaciones sobre el frente (pila), la version mas simple ya alcanza. Si se necesita insertar/eliminar por ambos extremos con costo bajo, la unica opcion realmente buena es DoublyLinkedListWithTail.
- **El puntero a tail solo ayuda a medias en una lista simple.** SinglyLinkedListWithTail arregla pushBack pero no popBack, porque sin puntero al nodo anterior no se puede actualizar tail sin recorrer. Esa es la razon de fondo por la que las listas dobles existen: el costo extra de memoria del puntero prev se paga con operaciones $O(1)$ en ambos extremos.
- **El arreglo circular no sirve para todo, sirve para colas.** Para un Stack, donde push/pop siempre trabajan sobre el mismo extremo, el arreglo circular y el dinamico rinden igual (se confirma en las graficas de 3.5, las curvas quedan superpuestas). Para un Queue la historia es distinta: el arreglo dinamico obliga a desplazar todo el arreglo en cada dequeue ($O(n)$), y el circular lo resuelve en $O(1)$ simplemente moviendo un indice. Es la diferencia mas grande que se vio en todo el taller.
- **Arreglos dinamicos vs listas enlazadas, en general:** los arreglos ganan en localidad de memoria y en operaciones sobre un extremo fijo (mas rapidos en la practica por el overhead de asignar nodos), pero pierden feo apenas hay que insertar o eliminar en el medio o en el extremo contrario al que optimizan (arreglo dinamico simple = $O(n)$ por el corrimiento de elementos). Las listas enlazadas no tienen ese problema porque nunca desplazan datos, solo repintan punteros, pero pagan con un nodo extra por elemento y peor localidad de cache.
- **Cuando usar cada una en la practica:** un Stack de arreglo dinamico (equivalente a usar ArrayDeque/ArrayList en Java) alcanza para casi cualquier caso real porque push/pop en un extremo son $O(1)$ de cualquier forma. Un Queue, en cambio, deberia implementarse siempre con arreglo circular (o con una lista doblemente enlazada con tail) y nunca con un arreglo dinamico simple: el resultado empirico de 3.6 deja claro que la version ingenua se vuelve inutilizable con colecciones grandes.